News Bias Detector

Zero Shot Zealots

Description

- Detect bias of news articles based on headlines and content
- Multi-class categorization: identifying if an article leans left, center, or right
- Experiment with different architectures and tokenization techniques
- Train on scraped data from AllSides
    - US-based news source that presents “all sides” of political events/topics/opinions

Previous Solutions

- Stanford NLP Model News Bias Detection
- Multi-Bias Detection with LLMs
- Recent Article on Media Bias Detection with DL

Data Preprocessing

Load Data

```
filename = "allsides_balanced_news_headlines-texts.csv"

import requests
url = 'https://raw.githubusercontent.com/irgroup/Qbias/refs/heads/main/allsides_balanced_news_headlines-texts.csv'
res = requests.get(url, allow_redirects=True)
with open(filename,'wb') as file:
    file.write(res.content)

df = pd.read_csv(filename)
print("Shape:", df.shape)
print(df.head(10))
print(f"Columns: {df.columns}")
```

```
Shape: (21754, 7)
```

Data Exploration

Data Exploration

```
# Explore number of labeled articles in each bucket
left_df = df[df["bias_rating"] == "left"]
right_df = df[df["bias_rating"] == "right"]
center_df = df[df["bias_rating"] == "center"]

print(f"Left: {left_df.shape}")
print(f"Right: {right_df.shape}")
print(f"Center: {center_df.shape}")
```

```
Left: (10275, 6)
Right: (7226, 6)
Center: (4253, 6)
```

Data Exploration

```
average_length_left = left_word_count / left_df.shape[0]
average_length_right = right_word_count / right_df.shape[0]
average_length_center = center_word_count / center_df.shape[0]

print("Average length of left-leaning articles:", average_length_left)
print("Average length of right-leaning articles:", average_length_right)
print("Average length of center-leaning articles:", average_length_center)
```

```
Average length of left-leaning articles: 64.11299270072993
Average length of right-leaning articles: 66.65942430113479
Average length of center-leaning articles: 70.83611568304725
```

Data Exploration

```
# Explore word counts across articles
left_word_count = sum(left_df["text"].fillna("").str.split().apply(len))
right_word_count = sum(right_df["text"].fillna("").str.split().apply(len))
center_word_count = sum(center_df["text"].fillna("").str.split().apply(len))

print("Left Leaning Articles Word Count:", left_word_count)
print("Right Leaning Articles Word Count:", right_word_count)
print("Center Leaning Articles Word Count:", center_word_count)
```

```
Left Leaning Articles Word Count: 658761
Right Leaning Articles Word Count: 481681
Center Leaning Articles Word Count: 301266
```

There is a discrepancy in the total word count of the articles labeled left, right, and center

Data Exploration

```
left_token_count = sum(len(tokenizer.tokenize(str(text))) for text in left_df["text"] if pd.notna(text))
right_token_count = sum(len(tokenizer.tokenize(str(text))) for text in right_df["text"] if pd.notna(text))
center_token_count = sum(len(tokenizer.tokenize(str(text))) for text in center_df["text"] if pd.notna(text))

print("Left-leaning articles token count:", left_token_count)
print("Right-leaning articles token count:", right_token_count)
print("Center-leaning articles token count:", center_token_count)
```

```
Left-leaning articles token count: 837527
Right-leaning articles token count: 620797
Center-leaning articles token count: 388367
```

Baseline Model

- Considerations: BERT, Open source LLMs like Llama, DeepSeek
- Settled on BERT
  - Open to changes
  - Current research uses BERT for simple tokenization, sentiment analysis, and classification for similar tasks

BERT

- Bidirectional Encoder Representations from Transformers
- Model for natural language processing developed by Google
  - Learns bi-directional representations of text
  - BERT family: other similar models
- One consideration is that BERT performs well on sentences, so article titles may not be as easy to classify alone

Training Plan

- Primary task is to finetune BERT (the base model)
  - Documentation: BERT is “intended to be fine-tuned on a downstream task”
- Use the BERT base model (uncased) on HF
  - BERT Autotokenizer
  - Adam Optimizer
- Focus on article bodies for now
  - Use article title, tags, and source for later analysis
- Experiment with transfer learning if time allows

Testing plan

- 70/15/15 train/test/validation splits
- Stratified test set to ensure accurate representation of the whole dataset
    - Currently we have a near-even split of right, left, and center leaning articles in each category
- We can scrape more if we need more data for our model
- Evaluate performance of base model alone on our data
- Evaluate performance of fine tuned (BERT) model on our data and compare

Goal: run recent articles from any news source and accurately classify bias

Comments

- Cross validation in case the data is not great
- In evaluation also check confidence scores
  - Check if something very simple is classified incorrectly with a high confidence – then model isn't great
- Embedded models on HF (maybe BERT) OR prompting